

Introduction to Python for Social Research

A Beginner-Friendly Workshop

Dwaipayan Roy

Department of Computational and Data Sciences



24th September, 2026

Welcome!

What This Session Is About?

- Using Python to work with survey data
- Describing data in simple, honest numbers
- Comparing groups - rural vs urban, men vs women
- Testing whether differences are **real** or **by chance**
- Graphical representation of data using charts
- Doing all of this in a way that can be repeated and shared

What This Session Is About?

- Using Python to work with survey data
- Describing data in simple, honest numbers
- Comparing groups - rural vs urban, men vs women
- Testing whether differences are **real** or **by chance**
- Graphical representation of data using charts
- Doing all of this in a way that can be repeated and shared
- Everything is hands-on from start to finish

What This Session Is About?

- Using Python to work with survey data
- Describing data in simple, honest numbers
- Comparing groups - rural vs urban, men vs women
- Testing whether differences are **real** or **by chance**
- Graphical representation of data using charts
- Doing all of this in a way that can be repeated and shared
- Everything is hands-on from start to finish

Python is just a tool. The real work is thinking carefully about what the data means.

Why Python?

What makes Python useful

- It is free and open
- It works on any computer
- It handles large datasets easily
- It makes charts easily

Why Python?

What makes Python useful

- It is free and open
- It works on any computer
- It handles large datasets easily
- It makes charts easily

What this means for research

- Results can be re-created
- Peers can check your work
- Reviewers can see exactly what you did
- You can update the analysis later

Why Python?

What makes Python useful

- It is free and open
- It works on any computer
- It handles large datasets easily
- It makes charts easily

What this means for research

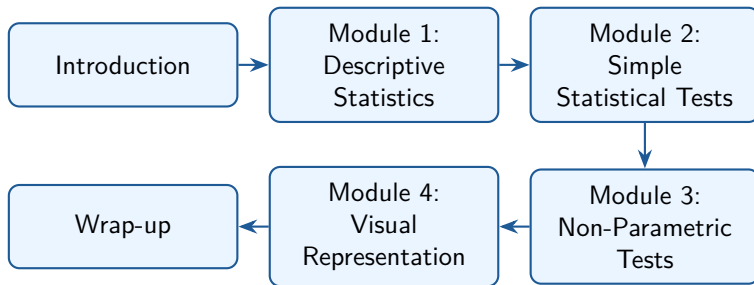
- Results can be re-created
- Peers can check your work
- Reviewers can see exactly what you did
- You can update the analysis later

With Excel, it is hard to handle large datasets, and results are often hard to reproduce.
With Python, everything is written down like a recipe - so anyone can cook the *same dish*.

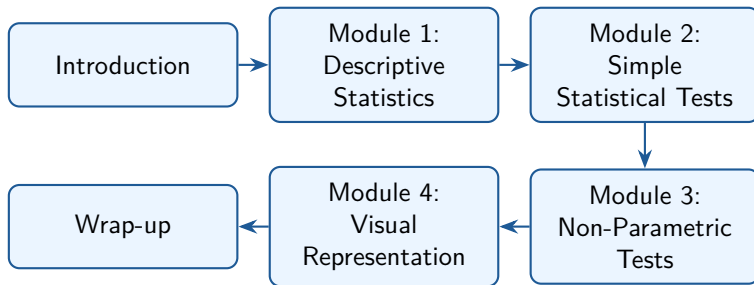
In the next 15-20 minutes, we will cover:

- Overview of the topics to be discussed
- Quick recap of Python
- The dataset we will work with

The Big Picture: What We Will Cover



The Big Picture: What We Will Cover



Each module builds on the one before it. We will use the same [dataset](#) throughout.

Descriptive statistics in simple steps:

- Opening a data file
- Looking at rows and columns
- Counting households in each group
- Calculating averages (mean and median)
- Comparing rural vs urban households
- Checking for missing values

Descriptive statistics in simple steps:

- Opening a data file
- Looking at rows and columns
- Counting households in each group
- Calculating averages (mean and median)
- Comparing rural vs urban households
- Checking for missing values

Descriptive statistics means describing the data before we test anything.

Testing whether differences are real:

- What is a p-value?
- Comparing two groups (t-test)
- Comparing three or more groups (ANOVA)
- Checking whether two variables move together (correlation)
- Predicting one variable from another (regression)

Testing whether differences are real:

- What is a p-value?
- Comparing two groups (t-test)
- Comparing three or more groups (ANOVA)
- Checking whether two variables move together (correlation)
- Predicting one variable from another (regression)

These tests tell us whether a difference we saw in the data is likely to be real, or just due to chance.

Non-parametric tests - for tricky data:

- Why income is often skewed
- Mann–Whitney instead of the t-test
- Wilcoxon for before-and-after comparisons
- Kruskal–Wallis instead of ANOVA
- Chi-square for two categorical variables
- Spearman for ranked correlation

Non-parametric tests - for tricky data:

- Why income is often skewed
- Mann–Whitney instead of the t-test
- Wilcoxon for before-and-after comparisons
- Kruskal–Wallis instead of ANOVA
- Chi-square for two categorical variables
- Spearman for ranked correlation

These are backup tools for when the standard tests do not fit the data.

Turning numbers into pictures:

- A simple bar chart
- A histogram of income
- A box plot to compare groups
- A scatterplot of education and income
- A line chart for change over time

Turning numbers into pictures:

- A simple bar chart
- A histogram of income
- A box plot to compare groups
- A scatterplot of education and income
- A line chart for change over time

A good chart shows the pattern at a glance.

Closing the workshop:

- Ethics of working with social data
- Why reproducibility matters
- How to share your analysis
- Where to go next
- Questions and discussion

Skills

- Opening and inspecting a dataset
- Calculating basic statistics
- Comparing groups
- Running simple tests
- Making clean charts

Habits

- Writing down every step
- Checking for missing values
- Reporting medians alongside means
- Saving work so it can be repeated
- Sharing analysis openly

`https://github.com/dwaipayanroy/Python-Workshop\_ISI`

Our Dataset

What it contains

- 200 households
- 9 simple variables
 - region, gender, age, education
 - income, household size
 - land ownership, SHG membership
 - wellbeing score (0–100)

Our Dataset

What it contains

- 200 households
- 9 simple variables
 - region, gender, age, education
 - income, household size
 - land ownership, SHG membership
 - wellbeing score (0–100)

Why a shared dataset?

- Everyone works with the same data
- Every code snippet will run
- You can adapt the code to your own data later
- The focus stays on method, not on wrangling

Our Dataset

What it contains

- 200 households
- 9 simple variables
 - region, gender, age, education
 - income, household size
 - land ownership, SHG membership
 - wellbeing score (0–100)

Why a shared dataset?

- Everyone works with the same data
- Every code snippet will run
- You can adapt the code to your own data later
- The focus stays on method, not on wrangling

The dataset is synthetic - it does not describe real households. But its patterns are realistic.

Setting The Environment!

Option A: Google Colab (no installation required)

- 1 Go to <https://colab.research.google.com>
- 2 Sign in with a Google account → *New Notebook*

Setting The Environment!

Option A: Google Colab (no installation required)

- 1 Go to <https://colab.research.google.com>
- 2 Sign in with a Google account → *New Notebook*

Option B: Local installation

```
# Using Anaconda Prompt / Terminal and type:  
python --version  
pip install pandas numpy scipy matplotlib seaborn statsmodels
```

Setting The Environment!

Option A: Google Colab (no installation required)

- 1 Go to <https://colab.research.google.com>
- 2 Sign in with a Google account → *New Notebook*

Option B: Local installation

Using Anaconda Prompt / Terminal and type:

```
python --version
```

```
pip install pandas numpy scipy matplotlib seaborn statsmodels
```

Verify Installation

```
import pandas as pd, numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt, seaborn as sns
print("All set! Pandas version:", pd.__version__)
```

Software

- `pandas` - works with tables
- `numpy` - does maths
- `scipy` - runs statistical tests
- `matplotlib` and `seaborn` - make charts
- `statsmodels` - runs regression

Software

- `pandas` - works with tables
- `numpy` - does maths
- `scipy` - runs statistical tests
- `matplotlib` and `seaborn` - make charts
- `statsmodels` - runs regression

You do not need to memorise any of these names. They are just the tools we will use.

Your First Python Command

```
print("Hello, world!")
```

```
Hello, world!
```

`print()` shows text on the screen. It is the simplest command in Python - and the first one everybody writes.

Loading the Tools

```
import pandas as pd
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns

print("All tools loaded!")
```

import means "bring in a tool". We do this once at the start of every notebook.

What We Will Cover

Basics

- 1 Importing tools
- 2 Storing values in names
- 3 Printing output
- 4 Simple maths
- 5 Comparisons
- 6 Combining conditions
- 7 Comments in code

Working with data

- 8 Opening a CSV file
- 9 Looking at the data
- 10 Selecting a column
- 11 Filtering rows
- 12 Combining conditions
- 13 Simple statistics
- 14 Common pandas commands

You do not need to learn all of these by heart. This deck is a reference - come back to it whenever you get stuck.

1. Importing Tools

`import` means "bring in a tool". Each tool does a different job.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

1. Importing Tools

`import` means "bring in a tool". Each tool does a different job.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

pandas - works with tables

numpy - does maths

matplotlib - makes basic charts

seaborn - makes prettier charts

scipy - runs statistical tests

1. Importing Tools

`import` means "bring in a tool". Each tool does a different job.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

pandas - works with tables

numpy - does maths

matplotlib - makes basic charts

seaborn - makes prettier charts

scipy - runs statistical tests

The `as pd` part gives each tool a short nickname. It saves typing.

2. Storing Values in Names

A **variable** is a name that holds a value. Think of it as a labelled box.

```
x = 5  
name = "Rural"  
income = 25000  
is_migrant = True
```

```
x          -> 5  
name       -> "Rural"  
income     -> 25000  
is_migrant -> True
```

The equals sign = means "store this value under this name".

3. Printing Output

`print()` shows a value on the screen. It is the simplest way to check what is inside a variable.

```
print("Hello, world!")  
print(5 + 3)  
print("Rural households:", 130)  
  
x = 42  
print("The answer is", x)
```

3. Printing Output

`print()` shows a value on the screen. It is the simplest way to check what is inside a variable.

```
print("Hello, world!")  
print(5 + 3)  
print("Rural households:", 130)  
  
x = 42  
print("The answer is", x)
```

```
Hello, world!  
8  
Rural households: 130  
The answer is 42
```

4. Simple Maths

Python does arithmetic just like a calculator. It follows BODMAS rule.

```
x = 10
x = x + 3      # addition
x = x - 3      # subtraction
x = x * 3      # multiplication
x = x / 3      # division
x = x ** 2     # power (10 squared)
x = x % 3      # remainder
x = (x + 3) * 2 # brackets first
```


4. Simple Maths

Python does arithmetic just like a calculator. It follows BODMAS rule.

```
x = 10
x = x + 3      # addition
x = x - 3      # subtraction
x = x * 3      # multiplication
x = x / 3      # division
x = x ** 2     # power (10 squared)
x = x % 3      # remainder
x = (x + 3) * 2 # brackets first
```

```
13
7
30
3.333...
100
1
26
```

5. Comparisons

A comparison asks a yes/no question. The answer is either True or False.

```
x = 5
x == 5      # is x equal to 5?
x != 5      # is x not equal to 5?
x > 3       # is x greater than 3?
x < 10      # is x less than 10?
x >= 5      # is x at least 5?
x <= 5      # is x at most 5?
```

```
True
False
True
True
True
True
```

Note the double equals sign: == compares, while = assigns.

6. Combining Conditions

We can combine two or more comparisons with `and`, `or`, and `not`.

```
x = 5
```

```
(x > 3) and (x < 10)    # both true?  
(x > 3) or  (x < 10)    # at least one true?  
not (x > 3)             # opposite
```

```
True  
True  
False
```

Wrap each comparison in brackets to keep the logic clear.

7. Comments in Code

A **comment** is a note to yourself. Python ignores it. Comments start with the `#` symbol.

```
# This is a comment. Python ignores it.
```

```
income = 25000    # monthly income in rupees
```

```
# We can use comments to explain a step:
```

```
income = income + (income / 10)
```

7. Comments in Code

A **comment** is a note to yourself. Python ignores it. Comments start with the `#` symbol.

```
# This is a comment. Python ignores it.  
  
income = 25000    # monthly income in rupees  
  
# We can use comments to explain a step:  
income = income + (income / 10)
```

Write comments for future use of the code!

8. Opening a CSV File

`pd.read_csv()` opens a file and loads it into a table. We call this table a **dataframe**.

8. Opening a CSV File

`pd.read_csv()` opens a file and loads it into a table. We call this table a **dataframe**.

```
import pandas as pd

df = pd.read_csv("village_survey.csv")

print(df)
```

- `"village_survey.csv"` - the file name in quotes
- `df` - the name we give to the table

8. Opening a CSV File

`pd.read_csv()` opens a file and loads it into a table. We call this table a **dataframe**.

```
import pandas as pd

df = pd.read_csv("village_survey.csv")

print(df)
```

- `"village_survey.csv"` - the file name in quotes
- `df` - the name we give to the table

The file must be in the same folder as your notebook. If not, you must give the full path.

8. Opening a CSV File in Google Colab

- Upload the file in the Google Drive

8. Opening a CSV File in Google Colab

- Upload the file in the Google Drive

```
from google.colab import drive
drive.flush_and_unmount() # Required if file uploaded after mounting
drive.mount('/content/drive')

# Then read it using its path in your drive
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/folder/filepath')
```

- Need to allow a bunch of Google drive access permissions (**all are revocable**).

8. Opening a CSV File in Google Colab

- Upload the file in the Google Drive

```
from google.colab import drive
drive.flush_and_unmount() # Required if file uploaded after mounting
drive.mount('/content/drive')

# Then read it using its path in your drive
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/folder/filepath')
```

- Need to allow a bunch of Google drive access permissions (**all are revocable**).

FileNotFoundError are common. Need to check the path carefully.

9. Looking at the Data

Before analysing anything, look at the data. Five commands give you a quick overview.

9. Looking at the Data

Before analysing anything, look at the data. Five commands give you a quick overview.

```
df.head()           # first 5 rows
df.tail(3)          # last 3 rows
df.shape            # (rows, columns)
df.columns.tolist() # column names
df.dtypes           # data type of each column
```

```
(200, 10)
['household_id', 'region_type', 'gender', 'age',
 'education_years', 'household_size', 'income',
 'land_ownership', 'shg_membership', 'wellbeing_score']
```

10. Selecting a Column

Use square brackets with the column name in quotes to select one column.

```
df['income']  
df['region_type']  
df['wellbeing_score']
```

This gives you a single column of data. You can then calculate things on it - like the mean or the median.

```
df['income'].mean()  
df['income'].median()
```

11. Filtering Rows

Use a condition inside square brackets to select only the rows that match.

```
# All rural households
rural = df[df['region_type] == 'Rural']

# All households with income above 30,000
high_income = df[df['income] > 30000]

# All female-headed households
female = df[df['gender] == 'Female']
```

The condition is written like a comparison. Python keeps only the rows where it is True.

12. Combining Conditions in Filtering

To filter on two conditions, use & for "and" and | for "or". Wrap each condition in brackets.

```
# Rural AND female-headed
df[(df['region_type'] == 'Rural') &
    (df['gender'] == 'Female')]

# Rural OR urban with high income
df[(df['region_type'] == 'Rural') |
    (df['income'] > 50000)]
```


13. Simple Statistics

Once you have a column of numbers, you can calculate basic statistics with single commands.

```
df['income'].mean()      # average
df['income'].median()    # middle value
df['income'].min()       # smallest
df['income'].max()       # largest
df['income'].std()       # spread
df['income'].count()     # how many (not missing)
```

14. Common pandas Commands

```
# Count households in each category
df['region_type'].value_counts()

# All summary numbers at once
df[['age', 'income', 'wellbeing_score']].describe()

# Mean of a number, grouped by a category
df.groupby('region_type')['income'].mean()

# Cross-tabulate two categories
pd.crosstab(df['region_type'], df['land_ownership'])

# Check for missing values
df.isnull().sum()
```

Common Beginner Errors

- `NameError: name 'df' is not defined`
 - You forgot to import pandas or open the file.
- `KeyError: 'Income' (with capital I)`
 - Column names are case-sensitive. Use `'income'`.
- `FileNotFoundError`
 - The CSV is not in the same folder as your notebook.
- `TypeError` when using `and` in filtering
 - Use `&` instead of `and` inside `df[...]`.

How to Debug

- Read the error message carefully - it usually says what is wrong
- Check the line number
- Ask: what did I type vs what did I mean to type?
- Check spelling and capital letters
- Check that the data file is in the right place

How to Debug

- Read the error message carefully - it usually says what is wrong
- Check the line number
- Ask: what did I type vs what did I mean to type?
- Check spelling and capital letters
- Check that the data file is in the right place

Debugging is not a sign of failure. It is the normal rhythm of programming.

You do not need to remember everything on these slides.

You only need to know **where to look** when you get stuck.

This deck is your reference. Come back to it any time.